CASE-STUDY PRACTICE | CAPSTONE 6 | STUDY EDITION

People You May Know: Graph Candidates, Two-Pass Ranking and Equity

Scenario questions on recommending connections from a social graph, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a graph, a model or a serving system: figures are the pages' or arithmetic from stated inputs, and member counts, degrees, row sizes and node speeds are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the problem: the objective, the labels and the size of the graph 6

2 Data and candidates: edge types, triangle closing and cold start 10

3 Ranking and equity: two passes, calibration, impression rules and fairness 15

4 Serving, evaluation and operations 20

Sources and confidence 25

Every question, by part

Part 1 Framing the problem: the objective, the labels and the size of the graph 6

01 What does PYMK predict? Framing 6

02 Size the edge table Estimation 7

03 Second-degree expansion Estimation 8

04 How many nodes does the daily batch need? Throughput 9

Part 2 Data and candidates: edge types, triangle closing and cold start 10

05 Why not one ranking model for every edge? Data 10

06 What does triangle closing find and miss? Candidates 11

07 A member with no connections Cold start 12

08 Review the friends-of-friends function Code review 13

Part 3 Ranking and equity: two passes, calibration, impression rules and fairness 15

09 Two passes, and why they are independent Architecture 15

10 Why calibrate, and what does it not fix? Calibration 16

11 Importance factors and impression guarantees Ranking 17

12 Discount the overloaded recipient Re-ranking 18

13 Read the fairness result Evaluation 19

Part 4 Serving, evaluation and operations 20

14 Why recompute everything every day? Serving 20

15 Cost one member's expansion Scale 21

16 Review the connection table Code review 22

17 Measure the recipient side Evaluation 23

18 Risks, and ten times the graph Risk 24

The case in one page

A summary of the case study to read before the questions.

Problem	Recommend people a member may know from a graph that grows faster than the site, without favoring members who are already well connected.
Core idea	A binary classifier with triangle closing as the main candidate signal, a re-ranker that discounts overloaded recipients, and a fairness re-ranker for infrequent members.
Wider system	A second-pass ranker orders cohorts of different entity types by predicted downstream interactions: likes, comments and re-shares, not accepted invitations (source-confirmed).
Scale	Hundreds of terabytes a day, hundreds of billions of potential connections, billions of samples, hundreds of features, daily refresh (source-confirmed, team page).
Headline results	Decay: overloaded recipients down 50%, requests down 1%, recipient sessions up 1%. Fairness: invitations sent to infrequent members up 5.44%, connections they made up 4.8%, frequent members neutral (source-confirmed).
Not published	The label definition, offline metrics, model sizes, latency and the re-ranking parameters.
First to break	Second-degree expansion, which the team page says grows much faster than the site.
Easy-to-miss traps	A downstream interaction is not a connection. A 1:2 impression guarantee gives follow cohorts more impressions. Edge halves are members times degree. Neutral is not proof of no harm.

Numbers to know

Each figure carries the tag that says how it was obtained. The sources are blog posts with few numbers, so member counts, degrees, row sizes and node speeds are illustrative.

Quantity	Value	Evidence
Scale statements	Hundreds of terabytes a day; hundreds of billions of potential connections; billions of samples; hundreds of features; results refreshed daily	source-confirmed (team page)
Edge rows	700,000,000 × 400 = 2.8 × 10^11 halves; counting × 2 again would give 5.6 × 10^11	illustrative; recomputed
Storage	2.8 × 10^11 × 24 B = 6.7 TB; 3.3 GB and 137 million rows per shard of 2,048	illustrative; recomputed
Second-degree walks	400 × 400 = 160,000 per member; 1.12 × 10^14 a day	illustrative; recomputed
Candidates	3 × 10^11 ÷ 7 × 10^8 = 430 per member, 0.27% of the walks; or degree sqrt(430) = 21	illustrative; recomputed
Batch load	2.8 × 10^11 lookups a day = 3.2 million a second; 896 TB returned; about 162 nodes at 20,000 a second	illustrative; recomputed
Impression decay results	Overloaded recipients 50% fewer; requests sent 1% fewer; recipient sessions 1% more; sender sessions neutral; 1% session win	source-confirmed (2021 post)
Fairness results	Invitations sent to infrequent members +5.44%; connections made by them +4.8%; frequent members neutral	source-confirmed (2021 post)
Random bucket	5% of traffic with cohorts in random order trains the second-pass ranker	source-confirmed (2020 post)
Impression guarantee example	1:2 importance connection to follow: twice as many follow cohorts; 12 cohorts = 4 and 8	source-confirmed (example); recomputed
Decay example	df = (20 − 12) ÷ 15 = 0.53: score 0.60 becomes 0.32, below an uncontested 0.40	illustrative; recomputed
Change fan-out	One new connection affects about 800 members; 0.1% new a day is about 160 changes in one member's 160,000-connection neighborhood	illustrative; recomputed
Hourly recompute	24 × 896 TB = about 21.5 PB a day	illustrative; recomputed
One-copy table	1.4 × 10^11 rows and 3.4 TB, against 2.8 × 10^11 and 6.7 TB with both directions	illustrative; recomputed
Ten times members	2.8 × 10^12 rows; 32 million lookups a second; about 1,620 nodes	illustrative; recomputed

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Candidate source	Friends of friends, then pair features	Add other sources if mutual connections miss people	source-confirmed; reasoned
Model	Logistic regression over hundreds of features	Non-linear models, which the 2021 post mentions	source-confirmed
Ranking across types	A second-pass ranker with calibration	One model for a single entity type	source-confirmed
Coupling	Independent first and second pass	Tight coupling if accuracy outweighs iteration speed	source-confirmed
Exposure for new types	Impression guarantees per viewer and edge type	Rely on the ranker alone if every type has plenty of data	source-confirmed
Fairness	Re-rank after scoring	Reweight the training data to correct the source	source-confirmed; reasoned
Graph storage	Both directions keyed by owner	One row per connection only if lookups by both ends are rare	reasoned
Recompute	Daily batch	Incremental only if a change touches a small part of the output	source-confirmed; reasoned

PART 1

Framing the problem: the objective, the labels and the size of the graph

Saying exactly what is predicted, separating events that sound alike, and sizing a graph and its second-degree expansion from stated assumptions.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Connection graph	A graph whose nodes are members and whose edges are connections. A connection is two-way, so each edge can be stored as two rows, one from each end.
P(connect)	The score PYMK gives a pair of members: an estimate of the propensity of the two to connect. Recommendations are ranked by it.
Invitation and acceptance	A connection forms only when a sender sends an invitation and the recipient accepts it. Invitations sent and connections made are separate counts.
Downstream interaction	In the 2020 post, a like, comment or re-share on content produced by a recommended entity. It is what the second-pass ranker predicts, and it is not an accepted invitation.
Degree	The number of connections a member has. The average degree times the number of members gives the number of stored edge halves.
Second-degree expansion	Looking at connections of connections. For a member with d connections who each have d connections, that is d times d walks.
Mean-field estimate	An estimate that uses the average of a quantity everywhere. With uneven degrees it understates a quantity that depends on squares.
Throughput-sized and capacity-sized	A system is capacity-sized when the data does not fit on one node, and throughput-sized when the request rate does not. Shard counts should follow whichever binds.
Where to read	LinkedIn Engineering: Building a Heterogeneous Social Network Recommendation System (2020) | LinkedIn Engineering: Optimizing PYMK for Equity in Network Creation (2021) | LinkedIn Engineering: People You May Know team page

01 What does PYMK predict?

FRAMING

SCENARIO

A colleague summarizes the LinkedIn pages: PYMK predicts connections, the second-pass ranker predicts connections too, and labels are connected pairs in the graph. Check each statement against the sources.

HOW TO THINK ABOUT IT

Read what each page says is predicted, then ask what is and is not published about labels.

Q What does the team page say?

A At the heart of PYMK is a binary classification problem: predict whether two people know each other. PYMK uses a logistic regression model to combine hundreds of features (source-confirmed, team page).

- The 2021 post says PYMK estimates the propensity to connect with a combination of linear and non-linear models and produces a P(connect) score. The pages differ in detail.

Q What does the second-pass ranker predict?

A Downstream interactions. A like, comment or re-share on content produced by the entity counts. For a connection edge, that is likes, comments or re-shares on content posted by the connection (source-confirmed, 2020 post).

- It is not an accepted invitation. The two should not be mixed up in a metric.

Q What do the pages say about labels?

A Nothing precise. The 2021 post says the algorithms learn from recommendations that result in successful matches, and that frequent members are more represented in the training data because they are more active.

- So training examples seem to come from outcomes of recommendations. The rule is not published, so stating it is a design choice.

Q How does a connection form?

A A sender sends an invitation, and the connection is formed only after the recipient accepts. Invitations sent and connections made are different metrics (source-confirmed, 2021 post).

- The post reports both, and they moved by different amounts in its results.

Q Reframe it.

A Quote what each source says is predicted, and keep similar words apart. A downstream interaction, an invitation and a connection are three different events.

Basis: The statements are from the three LinkedIn pages (source-confirmed); the point about unpublished labels is a statement about what the pages leave out.

02 Size the edge table

ESTIMATION

SCENARIO

You must store the connection graph as an edge table with one row per directed half, so that a member's connections come from one shard. Assume 700 million members and an average of 400 connections, rows of 24 bytes and 2,048 shards. A reviewer checks your arithmetic.

HOW TO THINK ABOUT IT

Count rows by the sum of degrees, and be careful about doubling.

Q How many rows?

A Each connection appears twice, once from each end. The number of halves is the sum of the degrees: 700,000,000 × 400 = 2.8 × 10^11 rows.

- Counting 700 million × 400 × 2 double counts, because 400 is already each member's own count of connections. That gives 5.6 × 10^11, twice too many.

Q How big is it?

A 2.8 × 10^11 × 24 bytes = 6.7 TB, about 3.3 GB on each of 2,048 shards, and about 137 million rows per shard.

- The pages give no member count or degree, so every figure here is an assumption.

Q Does capacity force 2,048 shards?

A No. 3.3 GB per shard is tiny, and 6.7 TB would fit on a few machines. Capacity alone would not need anywhere near that many.

- The shard count has to be justified by throughput (Question 4), or by wanting small units to move.

Q What would change with a different degree?

A Rows scale with degree. At an average of 100, 7 × 10^10 rows and 1.7 TB.

- The team page says the system looks at second-degree connections, which depends on the square of degree (Question 3).

Q Reframe it.
A Count a stored quantity by what is actually stored: halves of edges are the sum of degrees. A doubling in a formula should be one you can name.

Basis: The members, degree, row size and shard count are illustrative; arithmetic recomputed; the sources give no storage figures.

03 Second-degree expansion

ESTIMATION

SCENARIO

The team page says PYMK processes hundreds of billions of potential connections a day and that, because it looks at the second-degree network, processing grows much faster than the site. With 700 million members and 400 connections each, check whether that is consistent.

HOW TO THINK ABOUT IT

Count the second-degree walks, then compare with the page's candidate count.

Q How many second-degree walks are there?
A For one member, 400 friends with 400 connections each: 160,000. For all members, 700,000,000 × 160,000 = 1.12 × 10^14 a day.

- That is the mean-field count. With uneven degrees the sum of squared degrees is larger, so the real count is higher.

Q How does that compare with the page?
A Take 3 × 10^11 candidate pairs, within hundreds of billions. That is 3 × 10^11 ÷ 700,000,000 = about 430 candidates per member, which is 0.27% of the 160,000 walks.

- The walks are about 370 times more than the candidates.

Q What are the readings?
A Either members have about 21 connections each, since sqrt(430) = 21, which is far below typical. Or most second-degree pairs are removed before scoring.

- The second is more plausible. The pages do not say how the removal is done, for example by requiring several mutual connections or by capping.

Q What does growth faster than the site mean?
A Second-degree work grows with the square of degree and with the number of members. If members double and degree stays put, work doubles. If degree doubles as well, work grows by a factor of eight.

- The page states the pattern and no rate.

Q Reframe it.
A When two figures differ by orders of magnitude, ask what step sits between them. Here it is filtering, and the sources are silent on it.

Basis: The team page's statements are from the source (source-confirmed); the members, degree and 3 × 10^11 are illustrative; arithmetic recomputed; the filtering reading is reasoned.

04 How many nodes does the daily batch need?

THROUGHPUT

SCENARIO

A daily job expands friends of friends by reading each friend's connection list for each member. Using the Question 2 sizing, find the load, and say what sets the number of shards.

HOW TO THINK ABOUT IT

Count lookups, bytes returned and lookups per second, then divide by an assumed node speed.

Q How many lookups?

A One per edge row, because each member reads each friend's list: 2.8 × 10^11 a day, which is 3.2 million a second over 24 hours.

- Doing it in less than 24 hours raises the rate in proportion.

Q How much data comes back?

A Each lookup returns about 400 ids of 8 bytes: 2.8 × 10^11 × 400 × 8 = 896 TB.

- The team page says hundreds of terabytes a day, which is the same order. That agreement is a useful check, and it depends on the assumed degree.

Q How many nodes?

A At an assumed 20,000 lookups a second per node, 3.2 million a second needs about 162 nodes.

- The load is throughput-bound. Storage needs far fewer nodes.

Q So what are 2,048 shards for?

A Units of placement and movement. About 13 per node, small enough to rebalance.

- If a design reason for the count is only the row total, it is not a reason.

Q Reframe it.

A Compute the load in the unit the system is paid for in. Rows set capacity, lookups per second set the number of machines.

Basis: The hundreds of terabytes a day is from the team page (source-confirmed); the 20,000 lookups a second, 400 ids and 8 bytes are illustrative; arithmetic recomputed.

PART 2

Data and candidates: edge types, triangle closing and cold start

Why different edges need different signals, what triangle closing finds and misses, and what a graph feature cannot do for a member with no edges.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Heterogeneous network	A graph with more than one type of node or edge. LinkedIn's has connection, follow and subscribe edges.
Bidirectional and unidirectional edges	A connection gives both members access to each other's content. A follow or a subscribe gives only the follower access.
Triangle closing	If Alice knows Bob and Bob knows Carol, then maybe Alice knows Carol. The mutual friend is the signal that proposes a candidate.
Mutual connections	Members who are connected to both people in a pair. The count is a feature and an explanation shown to the member.
Candidate generation and scoring	First choose which pairs are worth scoring, cheaply. Then score those with richer features.
Impression discounting	Lowering the rank of results that a member has seen and ignored.
Hub	A member with a very large number of connections. Expanding a hub's list is expensive, and it connects to many people only weakly.
Cold start	A new member or item with no history, for whom history-based signals are empty.
Where to read	LinkedIn Engineering: Building a Heterogeneous Social Network Recommendation System (2020) | LinkedIn Engineering: Optimizing PYMK for Equity in Network Creation (2021) | LinkedIn Engineering: People You May Know team page

05 Why not one ranking model for every edge?

DATA

SCENARIO

A colleague proposes one model to recommend connections, hashtags to follow, groups and events, trained on all the interaction data together. The 2020 post separates them. Explain why, using the post's own points.

HOW TO THINK ABOUT IT

Compare the edge types, then look at what the training data over-represents.

Q How do the edge types differ?

A A connection is two-way: both the inviter and the receiver get access to each other's content. Follow and subscribe edges are one-way, giving the follower access to content from companies, hashtags, creators, groups, newsletters and events (source-confirmed, 2020 post).

- The edges serve different value propositions, so what counts as success differs.

Q What does the training data over-represent?

A Follow edges. Members tend to connect to a limited number of members and follow numerous hashtags, so follow edges dominate the data and get disproportionately high scores.

- In an extreme case a follow could be the default top suggestion even when connecting to a person would be more valuable.

Q What does the post do about it?

A It estimates the true value of an edge type with counterfactual experiments, and multiplies cohort scores by the resulting importance factors (Question 11).

- A single model trained on raw logs would inherit the bias.

Q What about the models themselves?

A The first-pass models are separate and independent. They can use XGBoost, logistic regression or deep networks, may optimize different business metrics, and may produce scores on different scales.

- Each team can iterate on its own model, and a second pass reconciles the scores (Question 9).

Q Reframe it.

A When the things you rank have different meanings and different data volumes, one model trained on the logs learns the volumes. Separate them, then reconcile.

Basis: The edge definitions, the over-representation and the independence of the models are from the 2020 post (source-confirmed).

06 What does triangle closing find and miss?

CANDIDATES

SCENARIO

A teammate says: triangle closing gives us candidates that are precise and easy to explain, so we should use only that. The team page says it then scores the closed triangles with other features. Say why.

HOW TO THINK ABOUT IT

Work out what two people with no mutual friend look like to this signal, and what the extra features add.

Q What is the signal?

A If Alice knows Bob and Bob knows Carol, then maybe Alice knows Carol (source-confirmed, team page). The pair has a mutual connection, which is also something to show the member.

- It is cheap and explainable.

Q What does it miss?

A Two people with no shared connection. Two colleagues about to start working together are invisible to it. Recall is low by construction.

- This follows from the definition. The pages do not report its recall.

Q What do the other features do?

A They rank the candidates: overlap at an organization or school, age difference and geographic distance. Triangle closing says which pairs are worth scoring, and these say how strong a match each is (source-confirmed, team page).

- The organizational overlap model uses when each person joined and left, the organization's size and how social it is.

Q Which candidate sources would you add?

A Shared organization or school with no mutual friend, and contacts imported by the member. These are reasoned additions, not from the pages.

- Measure the share of accepted invitations that came from each source.

Q Reframe it.

A A precise source with low recall needs partners. Say what it cannot see, and what feeds the pairs it never proposes.

Basis: The triangle closing description and the features are from the team page (source-confirmed); the recall point and the added sources are reasoned.

07 A member with no connections

COLD START

SCENARIO

A new member joins with no connections. Triangle closing finds nothing to expand. The 2020 post notes that inactive members lead to lack of data and lower-quality recommendations. Design the first week.

HOW TO THINK ABOUT IT

Say what signals exist on day one, and what each needs.

Q Why is there nothing to expand?

A Friends of friends starts from the member's friends. With none, the candidate set is empty. This follows from the mechanism and is not stated on the pages.

- The 2020 post says frequent members continuously provide rich data while inactivity leads to lower-quality recommendations (source-confirmed).

Q What signals exist on day one?

A Profile data: current and past organizations, schools, location and age. The team page names organization or school overlap, age difference and geographic distance as pair features.

- A pair can be scored with those even with no mutual connection, if it is proposed as a candidate.

Q How would candidates be proposed?

A From the same organization or school, or from imported contacts, as a design choice. Each needs the member to supply or confirm data.

- The pages do not describe how it is done.

Q What should you measure?

A Acceptance rate and the number of connections by the end of week one, for new members against others, and the share of new members with fewer than a few connections.

- The 2021 post is about members with few connections, whom it expects to gain more from each one.

Q Reframe it.

A A signal built from history is empty when history is empty. Name the first-day signals, and track the members who are served worst.

Basis: The 2020 post's statement is from the source (source-confirmed); the empty-set point and the first-week design are reasoned and untested.

08 Review the friends-of-friends function

CODE REVIEW

SCENARIO

A colleague writes the function below. In a test it returns the member's own id, people already connected to them, and a huge dictionary, and one run on a member who is a friend of a very popular account never finishes. Find the faults.

```
def friends_of_friends(member, adj):
    mutual = {}
    for friend in adj[member]:
        for other in adj[friend]:
            mutual[other] = mutual.get(other, 0) + 1
    return mutual
```

HOW TO THINK ABOUT IT

Walk one member through it, then count the work for a friend with a million connections.

Q What does it wrongly return?

A The member themselves, with a count equal to the number of friends, and existing connections, who are not candidates.

- Each friend's list contains the member, and each mutual friend's list contains the other friends.

Q What about a member not in the graph?

A adj[member] raises a KeyError for a new member. A new member has no edges (Question 7), and the function should return an empty result.

- Use adj.get(member, set()).

Q What about the popular account?

A Looking up a friend with 1,000,000 connections adds 1,000,000 steps to one member's expansion, against 400 × 400 = 160,000 for the typical walk, about 6 times a typical member's whole expansion, in one inner loop.

- Hubs connect to many people weakly, so their lists carry little signal per entry. Skip lists above a cap, such as 5,000.

Q What about the output?

A It returns every second-degree pair, which for 160,000 walks is tens of thousands of candidates. Most have one mutual friend.

- Keep a minimum mutual count and the top candidates by count, which also fits the page's point that far fewer pairs than walks reach scoring.

Q State the fix.

A Exclude the member and existing connections, return empty for unknown members, cap hub lists, and keep the top candidates with at least a minimum count.

- Test with a triangle, with a member with no friends, and with a hub above the cap.

Q Reframe it.

A In graph code, ask what the walk returns that is not a candidate, and what one very high-degree node does to the cost.

REFERENCE CODE

```
import heapq

def friends_of_friends(member, adj, hub_cap=5000, min_mutual=2, top=200):
    direct = adj.get(member, set())
    mutual = {}
    for friend in direct:
        friends = adj.get(friend, set())
        if len(friends) > hub_cap:                # skip hubs: weak signal, huge cost
            continue
        for other in friends:
            if other != member and other not in direct:
                mutual[other] = mutual.get(other, 0) + 1
    kept = [(n, other) for other, n in mutual.items() if n >= min_mutual]
    return {other: n for n, other in heapq.nlargest(top, kept)}
```

Basis: The sizes are illustrative; the faults and the fix are reasoned and the fix was checked by hand and not run.

PART 3

Ranking and equity: two passes, calibration, impression rules and fairness

How scores from different models are made comparable, how guarantees and importance differ, and how to read results about a re-ranker and a fairness change.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Edge-FPR and SPR	Edge-FPR: first-pass rankers, one set per edge or entity type, that rank entities among themselves. SPR: the second-pass ranker that ranks cohorts of different types against each other.
Cohort	A group of entities of the same type shown together as a horizontal carousel, such as people you may know or people who write about a hashtag.
Calibration	A mapping from a model's score quantiles to the observed response that trained it, so that a score means a known rate of that response.
Importance factor	A multiplier on a cohort's score that reflects the relative value of an edge type, estimated by counterfactual experiments.
Impression guarantee	A constraint on how many cohorts of each type a viewer is shown, so that new or low-scoring types still get exposure and gather data. The order among shown cohorts is still set by the ranker.
Random bucket	A small share of traffic, 5% in the post, where cohorts appear in random order. Its data trains the second-pass ranker without the bias of the first-pass ranking.
Impression decay	Multiplying a candidate's score by a factor between 0 and 1 that falls as the recipient receives more invitations.
Equality of opportunity and equalized odds	Fairness criteria. Equally qualified members of different groups should have equal true positive rates, and under equalized odds equal false positive rates too.
Where to read	LinkedIn Engineering: Building a Heterogeneous Social Network Recommendation System (2020) | LinkedIn Engineering: Optimizing PYMK for Equity in Network Creation (2021) | LinkedIn Engineering: People You May Know team page

09 Two passes, and why they are independent

ARCHITECTURE

SCENARIO

Seven teams each own a first-pass model for one entity type, and one team owns the second-pass ranker. A manager asks why the second pass is not trained end to end with the first-pass models, which might be a little more accurate.

HOW TO THINK ABOUT IT

State what each pass does, then count what coupling would force on every release.

Q What do the passes do?

A The first pass ranks entities of one type among themselves, with separate models that may use XGBoost, logistic regression or deep networks. The second pass ranks cohorts of different types against each other (source-confirmed, 2020 post).

- The second pass is an XGBoost model predicting the probability of downstream interactions with a cohort's top entities, using calibrated first-pass scores and member features.

Q What would coupling do?

A A marginal improvement to a first-pass model changes its score, which triggers retraining the second pass. An improvement to the second pass triggers retraining every first-pass model, the post says, with multiple back-and-forth updates.

- With seven first-pass teams releasing on their own schedules, the second pass would be retrained after nearly every release.

Q What does the post choose?

A To keep them completely independent, giving up some accuracy for iteration speed. The second pass is retrained on its own cadence.

- The post says tight coupling could give slightly better accuracy and slow the velocity of iteration.

Q How is the second pass trained?

A On a 5% random bucket where cohorts are shown in random order, so the first-pass ranking does not bias the data. The post suggests the first-pass teams can use the same data.

- The cost is that 5% of traffic sees a random order. If random order loses 20% of engagement (assumed), that is a 1% loss overall.

Q Reframe it.

A An interface between teams is a trade between accuracy and speed of change. Decide it on purpose, and pay for the loss with a calibration layer.

Basis: The two passes, the coupling argument and the random bucket are from the 2020 post (source-confirmed); the 20% engagement loss is illustrative and the 1% is arithmetic.

10 Why calibrate, and what does it not fix?

CALIBRATION

SCENARIO

The connection model gives a pair the score 0.40, and the hashtag-follow model gives a hashtag the score 0.40. The second pass must compare them. The post calibrates each first-pass score. Show what that does and what it leaves undone.

HOW TO THINK ABOUT IT

Use a small table of invented rates, then ask what each model's response variable is.

Q What is the problem with raw scores?

A They come from separate models that may optimize different business metrics and produce scores on different scales (source-confirmed). A 0.40 from one is not a 0.40 from the other.

- Ranking by raw scores would favor whichever model is more generous.

Q What does calibration do?

A It maps the quantiles of each first-pass score to the discretized observed response used to train that model. The mapping might be log-linear (source-confirmed).

- Example (invented): raw 0.40 from the connection model corresponds to 12% of recommendations being accepted. Raw 0.40 from the follow model corresponds to 31% being followed.

Q What is still different after calibration?

A The responses. 12% accepted invitations and 31% follows are different events, so the calibrated values are rates of different things.

- The post uses the calibrated scores as inputs to the second pass, which predicts one common target: downstream interactions.

Q Why does that target help?

A It puts every cohort on one outcome, likes, comments and re-shares on the recommended entity's content, so cohorts can be compared on it.

- It also means the second pass is not predicting connections. A change in downstream interactions is not a change in connections made.

Q Reframe it.

A Calibration makes a score mean something within one model. To compare across models you still need one common outcome to predict.

Basis: The calibration description and the second-pass target are from the 2020 post (source-confirmed); the rates of 12% and 31% are invented for illustration.

11 Importance factors and impression guarantees

RANKING

SCENARIO

A review note says: the post's 1:2 example means connections are weighted twice as much as follows. Check the note against the post.

HOW TO THINK ABOUT IT

Separate the two mechanisms, then compute the example.

Q What are the two mechanisms?

A Importance factors multiply the scores of cohorts, estimated from counterfactual experiments. Impression guarantees constrain how many cohorts of each type a viewer sees (source-confirmed, 2020 post).

- They answer different problems: the first corrects a biased score, and the second protects exposure.

Q What does the 1:2 example say?

A A 1:2 importance for connection against follow would ensure twice as many impressions of follow cohorts as of connection cohorts, while the ranking among them is still set by the second pass.

- So the note is backwards: follow cohorts get more impressions, and the number is a count, not a weight on a score.

Q Compute it.

A For a page of 12 cohorts, a 1:2 guarantee means 4 connection cohorts and 8 follow cohorts, with the order set by scores.

- An importance factor of 0.5 on a cohort with a score of 0.30 gives 0.15, a different kind of adjustment.

Q Why have guarantees at all?

A A new entity type could be pushed down and get too little exposure to gather training data. A guarantee prevents that.

- The guarantees are per viewer and per edge type, adjusted monthly or quarterly, and the post calls a global per-cohort guarantee far too complex to operationalize.

Q Reframe it.

A Read the unit of an example before you interpret it. A ratio of impressions and a weight on a score are different tools.

Basis: The mechanisms and the 1:2 example are from the 2020 post (source-confirmed); the 12 cohorts and the 0.5 factor are illustrative arithmetic.

12 Discount the overloaded recipient

RE-RANKING

SCENARIO

Influencers, executives and recruiters receive huge numbers of invitations. The 2021 post multiplies a candidate's score by a decay factor. Show how it changes a ranking, and say what the post reports and why it expected a loss.

HOW TO THINK ABOUT IT

Make a small decay function, apply it to two candidates, then read the results.

Q What is the rule?

A newScore = score × df, where df is between 0 and 1 and falls as the recipient receives more invitations. As a recipient gets more, the original score has to be higher to appear (source-confirmed, 2021 post).

- The post's example deprioritizes recipients who received more than 10 invitations in the past week. Production uses a piecewise linear function.

Q Apply it.

A Take df = 1 up to 5 invitations, then falling in a line to 0 at 20 (invented). Candidate A: score 0.60, 12 invitations: df = (20 − 12) ÷ 15 = 0.53, newScore 0.32. Candidate B: score 0.40, 2 invitations: df = 1, newScore 0.40.

- A was first. After the decay, B is first.

Q What did the team expect?

A The user experience to improve, and a small decrease in engagement because fewer invitations would be sent. It went ahead for the better experience (source-confirmed).

- Members inundated with requests get feed updates that seem random, and may miss relevant opportunities.

Q What happened?

A Overloaded recipients fell by 50%. Connection requests sent fell 1%. Sessions from the recipient side rose 1%. Sender-side sessions were neutral. Overall a 1% session win.

- The post's explanation: members with fewer invitations were receiving more, and invitations are more efficient at driving engagement for them.

Q Reframe it.

A A change that lowers a headline metric can raise the outcome that matters. Measure both sides, and state the expected loss before you launch.

Basis: The rule, the example, the expectation and the results are from the 2021 post (source-confirmed); the decay function and the two candidates are invented for illustration; arithmetic recomputed.

13 Read the fairness result

EVALUATION

SCENARIO

The 2021 post re-ranks connection suggestions for equality of opportunity and equalized odds. It reports a 5.44% rise in invitations sent to infrequent members, 4.8% more connections made by them, and neutral metrics for frequent members. A director says this proves nobody was harmed. Respond.

HOW TO THINK ABOUT IT

Read exactly what each number counts, then ask what the post gives for the neutral result.

Q What does each number count?

A Invitations sent to infrequent members, a recipient-side count, and connections made by infrequent members, a sender-side count. They are different metrics (source-confirmed).

- The post says it saw a 5.44% increase in invitations sent to IMs and a 4.8% increase in connections made by IMs.

Q How does the post read the neutral FM metrics?

A It says that shifting invitations from FMs to IMs would typically increase the metric for IMs and decrease it for FMs. Seeing neutral FM metrics indicates that recommendation quality has improved overall.

- It is the post's interpretation.

Q Why is that not proof of no harm?

A The post gives no confidence intervals or effect sizes for the neutral result. A test that cannot detect a small drop will report neutral.

- It also does not say whether the extra invitations to IMs were wanted, or what happened to other groups.

Q What would you ask for?

A The size of the effect on FMs with its interval, results by other cohorts, and acceptance and unfollow or complaint rates for the new invitations.

- The post says future work compares groups using protected attributes such as gender.

Q Reframe it.

A A neutral metric is a measurement with an error bar. Ask how small a change the test could have seen before you read neutral as nothing.

Basis: The results, the post's reading and the future work are from the 2021 post (source-confirmed); the questions are reasoned.

PART 4

Serving, evaluation and operations

Why recomputing daily can be the right call, what a sharded graph costs per member, and reading results and risks.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Batch and incremental computation	Batch recomputes everything on a schedule. Incremental recomputes only what a change affected.
Fan-out of a change	The number of results affected by one change. One new connection changes the candidates of every friend of both people.
Scatter-gather	Sending queries to several shards and combining the replies. A second-degree expansion needs one query for each friend's shard.
Self-join of an edge list	Joining the edge table with itself on the middle member to produce friends of friends in one distributed job, without per-member lookups.
Attribution	Crediting an outcome, here a recipient's session, to the sender whose action caused it.
Cannibalization	One kind of edge forming at the cost of another, so that a gain in one type is partly a loss in another.
Cohort-segmented metrics	Reporting the same metric separately for each group, so that an average dominated by a large group does not hide a small one.
Feedback loop	Recommendations shape behavior, behavior becomes training data, and the data shapes the next recommendations.
Where to read	LinkedIn Engineering: Building a Heterogeneous Social Network Recommendation System (2020) | LinkedIn Engineering: Optimizing PYMK for Equity in Network Creation (2021) | LinkedIn Engineering: People You May Know team page

14 Why recompute everything every day?

SERVING

SCENARIO

The team page says new PYMK results are pushed every day. An engineer proposes updating candidates incrementally as each connection forms. Size the proposal.

HOW TO THINK ABOUT IT

Count how many members' candidates one new connection can change.

Q What does a new connection change?

A The candidates of everyone who is a friend of either person: about 2 × 400 = 800 members, plus the two themselves.

- Triangle closing works on connections of connections, so a change travels one hop.

Q How many changes arrive a day?

A With 700 million members at 400 connections there are 1.4 × 10^11 connections. If 0.1% are new each day (assumed), that is 1.4 × 10^8 new connections.

- Each touches about 800 members' candidate lists.

Q What does one member see?

A Their own neighborhood holds about 160,000 connections. At 0.1% new a day, about 160 of them are new, so almost every member's candidates change every day.

- Incremental updates would touch nearly every member, one change at a time, with little saved.

Q What does the daily batch cost?

A About 896 TB of lookups returned (Question 4). Hourly recomputation would be 24 times that, about 21.5 PB a day.

- Daily is a middle point: a new connection is not reflected until the next run.

Q Reframe it.

A Incremental only pays when a change touches a small part of the output. Compute the fan-out before choosing it.

Basis: The daily cadence is from the team page (source-confirmed); the members, degree, 0.1% rate and 896 TB are illustrative; arithmetic recomputed.

15 Cost one member's expansion

SCALE

SCENARIO

Edges are stored in a table sharded by the owner member's id, with each connection stored twice. Cost one member's friends-of-friends expansion, and say how to avoid it.

HOW TO THINK ABOUT IT

Count queries, then look at the alternative that does not look members up one at a time.

Q How many queries?

A One for the member's own connections on their shard, and one per friend on the friend's shard. With 400 friends, 401.

- Friends' ids are spread over the id space, so these are nearly all on different shards.

Q What does that cost for the batch?

A 2.8 × 10^11 queries a day (Question 4). The member-by-member path pays one round trip per edge row.

- Round trips, not rows, are the cost.

Q What is the alternative?

A A self-join of the edge list on the middle member, as a distributed job, which is the shape of Hadoop-style workflows. The team page names Azkaban for Hadoop workflows, Voldemort and Kafka, and describes no sharded graph store.

- The join's raw output is the 1.12 × 10^14 walks, so counting mutual connections and dropping weak candidates must happen while the data is shuffled.

Q What does a hub do?

A A friend with a million connections returns a million ids in one reply, 6 times the whole typical expansion for one member. Cap or skip such lists.

- The cap trades a few strong signals for a large saving.

Q Reframe it.

A Count round trips, and ask whether a set-oriented job can replace a loop of lookups. A scatter that looks cheap per member is large over a whole population.

Basis: The infrastructure names are from the team page (source-confirmed); the sharded design, sizes and the 6 times are illustrative; arithmetic recomputed.

16 Review the connection table

CODE REVIEW

SCENARIO

A colleague stores each connection once, ordered as (low id, high id), and shards by the low id to halve storage. Looking up a member's friends is slow and sometimes incomplete in a test. Find the faults.

```
CREATE TABLE connections (
    low_id  BIGINT NOT NULL,
    high_id BIGINT NOT NULL,
    PRIMARY KEY (low_id, high_id)          -- sharded by low_id
);

-- friends of X
SELECT high_id FROM connections WHERE low_id = :x
UNION
SELECT low_id  FROM connections WHERE high_id = :x;
```

HOW TO THINK ABOUT IT

Work out which query finds the friends of a member who is the high id, and where those rows live.

Q What does a friends lookup need?

A Rows where the member is the low id, and rows where the member is the high id. That is two queries unioned.

- Half of a member's friends are on each side, on average.

Q Where do the rows live?

A Rows with the member as the low id are on the member's shard. Rows with the member as the high id are on the shard of the other person's low id, so they could be on any shard.

- Finding them means asking every shard, since no index has the high id.

Q What does the table save?

A Half the rows: 1.4 × 10^11 instead of 2.8 × 10^11, 3.4 TB instead of 6.7 TB at 24 bytes.

- It is a small saving against a lookup that hits every shard.

Q State the fix.

A Store both directions, keyed and sharded by the owner member, so that a member's friends come from one shard with one index lookup.

- Keep the two halves consistent when a connection forms or is removed, in one transaction or one idempotent write.

Q Reframe it.

A Shape storage for the lookup you do most. Saving half the rows is not a saving if it makes the main query touch every shard.

REFERENCE CODE

```
CREATE TABLE connections (
    owner_id  BIGINT NOT NULL,
    friend_id BIGINT NOT NULL,
    PRIMARY KEY (owner_id, friend_id)      -- sharded by owner_id
);

-- one row per direction: (A, B) and (B, A)
-- friends of X: a single-shard lookup
SELECT friend_id FROM connections WHERE owner_id = :x;
```

Basis: The sizes are illustrative; the faults and the fix are reasoned and were checked by hand and not run.

17 Measure the recipient side

EVALUATION

SCENARIO

The team says a change that sends fewer invitations improved overall sessions. A reviewer asks how they could attribute sessions of people who received invitations from several senders, and what the pages say about offline metrics.

HOW TO THINK ABOUT IT

Follow the post's attribution rule on its two examples, then read what is missing.

Q Why is the recipient side hard?

A Some recipients receive invitations from multiple senders, so it is hard to attribute the effect to a particular sender (source-confirmed, 2021 post).

- A sender-side metric is read directly from the A/B test, and the recipient side is not.

Q What is the rule?

A If the recipient gets a notification that Sender A wants to connect and then comes to accept, the session is attributed to A. If the recipient comes on their own and goes straight to Sender B's invitation, it is attributed to B.

- It is an attribution framework the team built.

Q What is the total effect of the decay change?

A Requests sent fell 1%, recipient sessions rose 1%, sender sessions were neutral: a 1% session win overall.

- Without the attribution, the sender side alone would have looked like a loss.

Q What do the pages say about offline metrics?

A None of the pages names an offline metric for the model. The evidence reported is from A/B tests.

- Say so, and do not invent one.

Q How do you monitor groups?

A Report the standard metrics, invitations, connections and sessions, for each cohort separately. An average over all members is dominated by frequent members.

- The post does not define a single equity metric.

Q Reframe it.

A Measure the side of a change where the benefit lands, and build the attribution to see it. A result read from one side can show the opposite sign.

Basis: The attribution rule, the results and the absence of an offline metric are from the pages (source-confirmed); the monitoring practice is reasoned.

18 Risks, and ten times the graph

RISK

SCENARIO

A launch review asks what can go wrong with PYMK and what grows if the graph grows ten times.

HOW TO THINK ABOUT IT

List the risks the pages name, then scale each quantity and say which grows with which.

Q What risks do the pages name?

A A feedback loop that favors frequent members. Overloaded recipients. Cannibalization across edge types. Lower-quality recommendations for inactive members (source-confirmed).

- Cold start for a new member follows from the mechanism.

Q What does ten times members do?

A Edge rows go from 2.8 × 10^11 to 2.8 × 10^12 and lookups to 32 million a second, about 1,620 nodes at 20,000 a second.

- It yields to machines, with the round-trip cost per member unchanged.

Q What does ten times degree do?

A Second-degree walks grow with the square: 100 times. The team page says processing grows much faster than the site, which fits this case.

- Filtering before scoring, and caps on hubs, become more important.

Q What does not scale with traffic?

A The fairness risk: it is a property of the training data and the objective, so it persists at any scale.

- Monitor it by cohort at any size.

Q Reframe it.

A Sort the quantities by what they scale with. Counts scale with members, second-degree work scales with the square of degree, and bias scales with neither.

Basis: The risks are from the pages (source-confirmed); the 10 times figures are illustrative arithmetic.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #6. The sources are three LinkedIn Engineering pages: the 2020 post on heterogeneous recommendations, the 2021 post on equity, and the PYMK team page, all read in full on 2026-10-17. They give a few qualitative scale statements, a handful of A/B results and no formulas or code. Everything about storage, sharding and serving paths in this book is a design argument made here and was not run on real systems.

- LinkedIn Engineering: Building a Heterogeneous Social Network Recommendation System, October 6, 2020, read in full 2026-10-17
https://www.linkedin.com/blog/engineering/optimization/building-a-heterogeneous-social-network-recommendation-system
- LinkedIn Engineering: Optimizing People You May Know (PYMK) for Equity in Network Creation, August 5, 2021, read in full 2026-10-17
https://www.linkedin.com/blog/engineering/member-customer-experience/optimizing-pymk-for-equity-in-network-creation
- LinkedIn Engineering: People You May Know team page, read in full 2026-10-17
https://engineering.linkedin.com/teams/data/artificial-intelligence/people-you-may-know